

CloudShield AI / DAIL

Technical Software Engineering & Final Implementation Specification

Version 1.0 | Implementation Baseline | 25 August 2026

Document purpose: This is the final engineering-oriented implementation plan for building the CloudShield AI / DAIL research prototype. It focuses on software architecture, technologies, interfaces, data, implementation order, testing, evidence, operational boundaries, and engineering acceptance criteria. Research motivation and prior-art material are treated as upstream decisions rather than re-designed here.

Document Control

Item	Decision
Document status	FINAL IMPLEMENTATION BASELINE
Primary system	CloudShield AI with DAIL as the research core
Initial implementation target	Deterministic DAIL core using fixed Terraform candidate patches
MVP cloud scope	AWS
MVP IaC scope	Controlled Terraform/HCL subset
Primary language	Python 3.12
Core persistence	SQLite initially; persistence abstraction retained
Graph	NetworkX
Schema validation	Pydantic v2
Testing	pytest
CI	GitHub Actions
LLM	Provider-agnostic adapter; introduced after deterministic core

1. Executive Engineering Decision

The system will be implemented as a deterministic, modular DAIL engine first and integrated with an LLM only after the deterministic fixed-patch path is proven. The implementation will preserve a hard trust boundary: the LLM proposes changes, DAIL evaluates candidates, and only the Promotion Controller can change Trusted State.

The first research-complete software milestone is not a UI, API, or live LLM. It is the fixed-patch end-to-end DAIL pipeline:

Trusted State → Candidate State Builder → Candidate State → Identity → Dependency → Impact → Verification → Promotion

The canonical safety property is that a security fix must not be promoted if it breaks a previously protected functional invariant. Conversely, when the security problem is fixed and the protected functional property is preserved, promotion should occur.

2. System Scope and Boundaries

2.1 In Scope

- CloudShield AI product context only insofar as it hosts and exercises DAIL.
- DAIL deterministic core.
- Stateful trusted-state management.
- Protected invariant ledger.
- Terraform normalization for the controlled subset.
- Resource identity and change classification.
- Supported dependency graph construction.
- Impact/invalidation analysis.
- Layered verification.
- Promotion decision logic.
- Fixed Terraform patch execution.
- LLM remediation adapter after deterministic core completion.
- Immutable research evidence capture.
- A/B/C experiment harness after core completion.
- Independent ground-truth/oracle path.

2.2 Supported Terraform/AWS Resource Scope

Resource	Role in MVP
aws_vpc	Network boundary
aws_subnet	Network segment
aws_route_table	Routing
aws_route_table_association	Subnet routing association
aws_internet_gateway	Internet connectivity
aws_instance	Compute
aws_security_group	Security policy
aws_security_group_rule	Explicit security rule
aws_db_instance	RDS database

2.3 Out of Scope

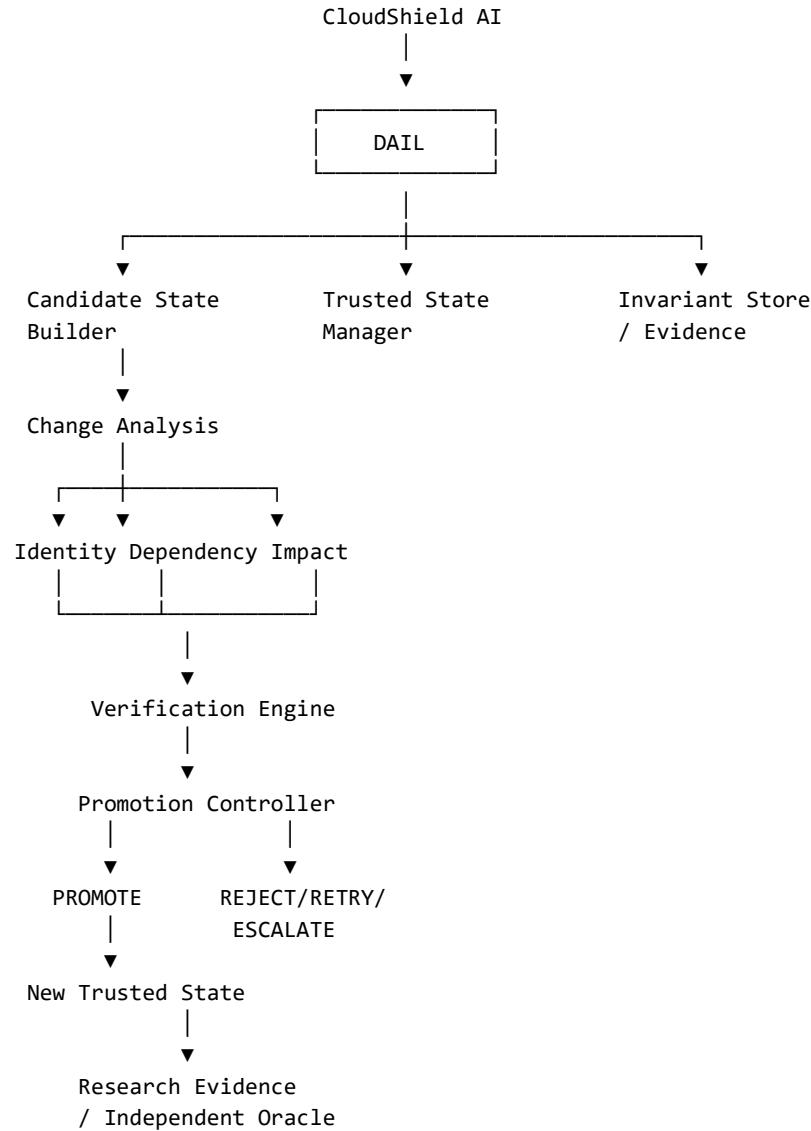
- All-AWS service coverage.
- Multi-cloud infrastructure.
- Kubernetes.
- Generic support for arbitrary Terraform constructs.
- Production-scale distributed architecture.
- Full CloudShield UI/productization before the research core is stable.
- Autonomous unrestricted cloud deployment.
- Treating unsupported infrastructure semantics as trusted.

3. Engineering Invariants

Invariant	Implementation Rule
Trust boundary	LLM output and candidate states are untrusted until promotion.
Trusted state mutation	Only the Promotion Controller may commit a candidate as trusted.
Rejected candidate	Never becomes the trusted baseline for the next repair

	attempt.
UNKNOWN	Never silently converts to PASS.
UNSUPPORTED	Must remain explicit and cannot silently become trusted proof.
Verifier error	Must remain distinguishable from property failure.
Affected	Means evidence is insufficient/current proof must be re-established; it is not equivalent to violated.
Identity uncertainty	Historical evidence cannot silently transfer across uncertain correspondence.
Dependency uncertainty	Must be handled conservatively.
Evidence freshness	Protected invariant evidence must be current for the trusted state it protects.

4. System Architecture



4.1 Component Responsibilities

Component	Primary responsibility	Must not do
Candidate State Builder	Construct a candidate representation from a fixed patch or LLM output.	Commit trusted state.
Trusted State Manager	Persist/version trusted and candidate snapshots.	Make verification decisions.
Invariant Store	Persist invariant definitions, scope, status, and evidence links.	Assume stale evidence is valid.
Identity Engine	Determine resource correspondence and change class.	Invent semantic equivalence when uncertain.
Dependency Engine	Construct supported typed dependency relationships.	Claim complete AWS semantics.
Impact Engine	Determine which obligations may require re-verification.	Declare that an invariant is actually valid.
Verification Engine	Evaluate candidate properties.	Promote state.
Promotion Controller	Apply promotion policy and commit/reject candidate.	Perform hidden verification logic.
LLM Adapter	Generate untrusted remediation proposals.	Directly mutate trusted state.
Evidence Pipeline	Persist traceable research events.	Rewrite historical evidence silently.
Independent Oracle	Provide evaluation path separate from DAIL verifier.	Reuse DAIL's own decision as ground truth.

5. End-to-End Software Workflow

1. Load the current Trusted State and its protected invariant/evidence set.
2. Receive a candidate Terraform patch. During the deterministic phase this is a fixed patch; later it may come from the LLM adapter.
3. Validate and construct Candidate State without mutating Trusted State.
4. Normalize candidate Terraform resources into the common internal model.
5. Compute resource identity and resource-level changes.
6. Build/update the supported dependency graph.
7. Determine change impact and affected protected invariants.
8. Run required structural, security, and functional verification.
9. Classify results using explicit result semantics.
10. Apply Promotion Controller policy.
11. If PROMOTE, atomically commit Candidate State as the next Trusted State and persist current evidence.
12. If REJECT, preserve the existing Trusted State and record the candidate/decision.
13. If RETRY or ESCALATE, preserve trust state and record the unresolved condition.
14. Persist a complete evidence trail for the transition.
15. When experimentation begins, independently evaluate outcomes through the ground-truth path.

6. Technology Stack

Layer	Final recommendation	Reason	Alternative
Language	Python 3.12	Research velocity, AWS/Terraform ecosystem, testing/data tooling, clear domain modeling.	Go, TypeScript

Project/dependency tooling	uv + pyproject.toml	Fast reproducible Python environments and modern packaging.	Poetry, pip
Schemas	Pydantic v2	Explicit contracts, validation, serialization, readable domain models.	dataclasses, attrs
Terraform interaction	Terraform CLI + HCL tooling	Terraform remains authoritative for supported structural interpretation; avoids rebuilding Terraform semantics.	Custom parser only
Persistence	SQLite initially via SQLAlchemy	Zero-infrastructure, transactional, reproducible, adequate for research MVP.	PostgreSQL
Graph	NetworkX	Mature, deterministic, easy graph inspection and testing.	Neo4j
AWS	boto3	Official Python AWS SDK and controlled runtime integration.	AWS CLI only
Testing	pytest	Strong fixture/parameterization ecosystem and suitable unit/integration/e2e organization.	unittest
API	FastAPI later	Useful external interface after core is stable; not a dependency of M1.	Flask
LLM	Provider-agnostic adapter	Prevents provider coupling and permits deterministic test adapters.	Direct provider SDK
Logging	Python structured logging approach	Debugging and operational traceability without conflating logs with research evidence.	Third-party logging stack
CI	GitHub Actions	Simple reproducible CI integrated with repository.	GitLab CI
Containerization	Docker optional/later	Useful for reproducible integration environments; unnecessary for initial deterministic core.	None initially

6.1 Technology Decision Rules

- Prefer the simplest technology that satisfies the research prototype.
- Do not introduce infrastructure because a production system might eventually need it.
- Keep domain interfaces independent of storage, LLM provider, and web framework.
- Use deterministic local fixtures wherever AWS access is not scientifically required.
- Every technology that can affect experimental validity must be documented before the main experiment.

7. Repository Architecture

```
cloudshield-ai/
├── pyproject.toml
├── README.md
├── .gitignore
├── .python-version
├──
├── dail/
│   ├── __init__.py
│   ├── state/
│   ├── invariants/
│   ├── terraform_model/
│   ├── identity/
│   ├── dependency/
│   ├── impact/
│   ├── verification/
│   ├── promotion/
│   ├── llm/
│   └── evidence/
├── experiments/
│   ├── scenarios/
│   ├── baselines/
│   ├── oracle/
│   ├── harness/
│   └── analysis/
├── terraform/
│   ├── fixtures/
│   └── modules/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   ├── adversarial/
│   └── regression/
├── scripts/
├── configs/
└── docs/
    ├── architecture/
    ├── decisions/
    ├── experiments/
    └── implementation/
```

7.1 Module Dependency Rule

Lower-level domain modules must not import higher-level orchestration modules. The core domain must not depend directly on FastAPI, a specific LLM provider, or a concrete database implementation. Infrastructure adapters sit at the edges.

8. Core Domain Model

8.1 Trusted State

TrustedState represents the only infrastructure snapshot eligible to serve as the baseline for the next repair attempt.

```
TrustedState
- state_id
- version
- terraform_snapshot
- normalized_resources
- protected_invariant_ids
- evidence_ids
- created_at
- parent_state_id
- status = TRUSTED
```

8.2 Candidate State

```
CandidateState
- candidate_id
- parent_trusted_state_id
- patch_id
- terraform_snapshot
- normalized_resources
- build_status
- created_at
- source = FIXED_PATCH | LLM
```

Candidate State is immutable after construction for the purposes of a single evaluation attempt. If a new patch is proposed, a new candidate is created.

8.3 Invariant

```
Invariant
- invariant_id
- name
- category = SECURITY | FUNCTIONAL
- predicate_definition
- scope
- status
- protected_since_state
- current_evidence_id
```

8.4 Evidence

```
VerificationEvidence
- evidence_id
- state_id
- invariant_id
- verifier_id
- verifier_version
- result
- details
- timestamp
```

- input_fingerprint

9. Terraform Normalized Model

All downstream DAIL components consume a normalized representation. They should not each implement their own Terraform parsing logic.

```
NormalizedResource
- resource_id
- terraform_address
- resource_type
- provider
- attributes
- references
- source_metadata
- state_metadata
- canonical_hash
```

9.1 Canonicalization

- Normalize resource addresses and resource types.
- Normalize equivalent security-group rule representations where supported.
- Normalize attribute ordering before hashing/serialization.
- Represent references explicitly.
- Preserve source/provenance information.
- Mark unsupported constructs explicitly.
- Never drop information merely because a downstream verifier does not use it.

9.2 Terraform Authority

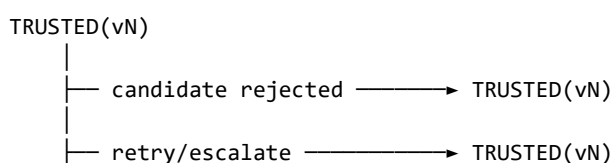
Terraform CLI remains the authoritative structural tool where practical. A custom normalization layer translates supported Terraform information into DAIL's stable internal model; it must not attempt to reproduce all Terraform semantics.

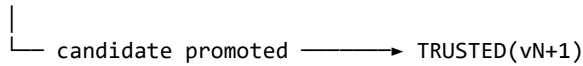
10. Trusted State Manager

10.1 Responsibilities

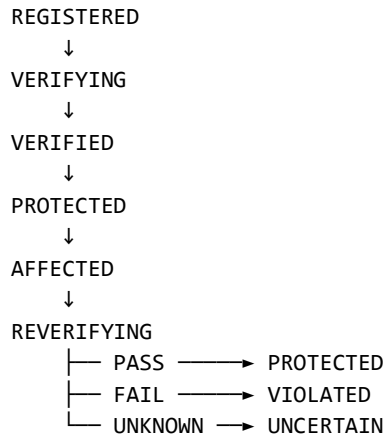
- Create and load trusted-state snapshots.
- Assign monotonically traceable versions.
- Associate protected invariants and current evidence.
- Create candidate evaluation contexts.
- Atomically commit only approved candidates.
- Prevent rejected candidates from replacing the trusted baseline.

10.2 State Transition





11. Invariant Store and Lifecycle



The exact status representation may be implemented as a domain enum plus evidence-backed transition rules. The key engineering rule is that protection is evidence-backed and state-specific.

12. Resource Identity Engine

12.1 Identity Output

SAME
DIFFERENT
UNCERTAIN

12.2 Change Output

UNCHANGED
MODIFIED
CREATED
DELETED
REPLACED
UNCERTAIN

12.3 Engineering Rules

- Use stable Terraform/resource addressing where it provides valid identity evidence.
- Use controlled canonical attributes for supported replacement/equivalence decisions.
- Treat ambiguous correspondence as UNCERTAIN.
- Do not transfer protected evidence across uncertain identity correspondence.
- Record identity rationale/provenance so decisions can be audited.

13. Dependency Graph Engine

13.1 Graph Model

Node = normalized Terraform resource or supported semantic object

Edge = typed supported relationship

Edge metadata = source/provenance + confidence/uncertainty

13.2 Dependency Categories

- Terraform structural references.
- Supported infrastructure relationships.
- Invariant-semantic relationships.

The MVP graph is intentionally incomplete with respect to all AWS semantics. Incompleteness must be explicit rather than hidden.

14. Impact / Invalidation Engine

14.1 Purpose

Impact analysis determines what may need re-verification. It does not itself prove whether an invariant still holds.

```
Changed resources
  ↓
Identity/change classification
  ↓
Direct scope intersection
  ↓
Supported dependency expansion
  ↓
Invariant relationship intersection
  ↓
Affected invariant set
```

14.2 Conservative Conditions

- Modified protected resource.
- Created/deleted/replaced resource in invariant scope.
- Supported dependency path from changed resource to protected invariant scope.
- Identity uncertainty affecting an invariant.
- Dependency uncertainty affecting an invariant.
- Supported restructuring that invalidates prior evidence assumptions.

A protected invariant that is affected must not retain the meaning of current proof until it is re-verified.

15. Verification Engine

15.1 Architecture

```
VerificationEngine
├─ StructuralVerifier
├─ SecurityVerifier
```

└─ FunctionalVerifier
└─ OptionalFormalVerifier

15.2 Result Contract

Result	Meaning	Promotion consequence
PASS	Required property established for this candidate.	May satisfy that verification requirement.
FAIL	Property is established as violated.	Blocking unless policy explicitly says otherwise; protected-invariant failure blocks promotion.
UNKNOWN	Verifier cannot establish the property.	Must not become PASS; normally blocks promotion where proof is required.
UNSUPPORTED	Verifier does not support the required construct/property.	Cannot silently count as proof.
VERIFIER_ERROR	Verification infrastructure failed or produced an execution error.	Must remain distinct from FAIL; policy determines retry/escalation.

15.3 Verification Layers

- Structural: Terraform/configuration validity and supported model consistency.
- Security: explicit security predicates for the MVP.
- Functional: selected supported reachability and functional properties.
- Optional formal: only where justified by a concrete verification need.

16. Promotion Controller

16.1 Decision Inputs

- Candidate structural validity.
- New remediation requirement result.
- Affected protected invariant set.
- Verification results for all required obligations.
- New-resource baseline security checks.
- Identity results.
- Dependency results.
- Blocking uncertainty/error conditions.

16.2 Decision Algorithm

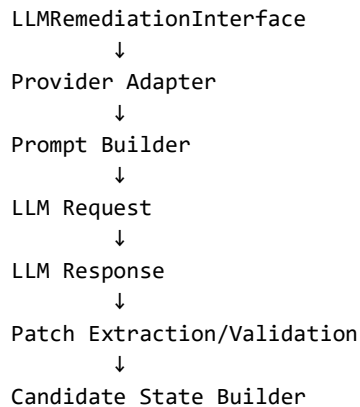
```
if candidate_structurally_invalid:  
    REJECT  
  
if remediation_requirement_not_satisfied:  
    REJECT  
  
if any_required_protected_invariant_not_verified_pass:  
    REJECT or RETRY/ESCALATE depending on result  
  
if required_identity_or_dependency_information_is_uncertain:  
    REJECT or ESCALATE according to policy  
  
if new_resources_fail_applicable_baseline_security:  
    REJECT
```

```
otherwise:
    PROMOTE
```

The exact policy table must be encoded as deterministic rules and covered by unit tests. The controller is the single authoritative mutation boundary.

17. LLM Integration

17.1 Adapter Boundary

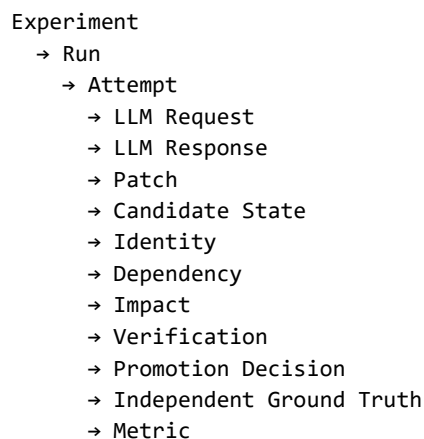


17.2 LLM Rules

- LLM output is always untrusted.
- LLM cannot call the Trusted State Manager directly.
- Every LLM proposal becomes a separately identifiable attempt.
- Malformed/unsupported patches fail candidate construction rather than being trusted.
- Provider details remain outside DAIL domain logic.
- A deterministic fake/test adapter must exist for tests.

18. Research Evidence and Audit Trail

18.1 Event Chain



18.2 Evidence Rules

- Evidence is append-oriented and traceable.
- Every decision references the state/candidate that produced it.
- Verifier version/configuration is recorded.
- Input fingerprints allow reproducibility checks.
- Research evidence is separate from transient debug logs.
- Historical evidence is not overwritten to make a result look cleaner.

19. Database / Persistence Design

19.1 Initial Persistence

SQLite is the initial persistence backend. SQLAlchemy should provide a persistence abstraction so the domain does not depend on SQLite-specific APIs.

19.2 Logical Entities

Entity	Purpose
states	Trusted/candidate snapshots and lineage
resources	Normalized resource records/fingerprints
invariants	Invariant definitions and lifecycle
evidence	Verification evidence
dependencies	Typed graph edges and provenance
impact_results	Affected-set computations
verification_results	Verifier outputs
decisions	Promotion outcomes
runs	Experiment/run metadata
attempts	Repair attempts and candidate lineage
llm_events	LLM request/response metadata
oracle_results	Independent ground-truth outcomes

20. API and Interface Contracts

20.1 Core Service Interfaces

```
CandidateBuilder.build(patch, trusted_state) -> CandidateState | BuildFailure
```

```
IdentityEngine.compare(trusted, candidate) -> IdentityReport
```

```
DependencyEngine.build(candidate) -> DependencyGraph
```

```
ImpactEngine.compute(trusted, candidate, graph) -> ImpactReport
```

```
VerificationEngine.verify(candidate, requirements) -> VerificationReport
```

```
PromotionController.decide(context) -> PromotionDecision
```

```
TrustedStateManager.commit(candidate, decision) -> TrustedState
```

These signatures are conceptual contracts for implementation. Exact class names and serialization details can be refined during M1 without changing the architectural responsibility boundaries.

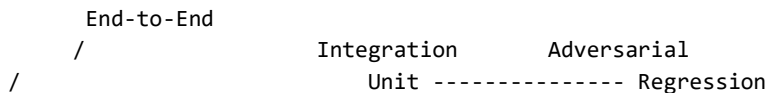
21. Error and Uncertainty Model

Condition	System behavior
Malformed Terraform patch	Candidate build failure; no trusted-state mutation.
Unsupported Terraform construct	Explicit UNSUPPORTED/UNKNOWN path; no silent trust.
Identity uncertain	Conservative invalidation/evidence handling.
Dependency uncertain	Conservative impact handling.
Verification UNKNOWN	Cannot satisfy a proof requirement.
Verifier error	Record error; RETRY/ESCALATE/REJECT according to policy.
Promotion rejection	Trusted state remains unchanged.
Storage failure during promotion	No partial trusted-state mutation; transactional failure.
LLM timeout/error	Attempt recorded; trusted state unchanged.

22. Security Engineering

- Treat all LLM output as untrusted input.
- Validate Terraform syntax and supported constructs before analysis.
- Never execute arbitrary LLM-generated infrastructure against production AWS.
- Use isolated controlled AWS environments for runtime tests.
- Store credentials outside source control.
- Use least-privilege AWS credentials for any runtime validation.
- Redact secrets from evidence and logs.
- Make candidate/trusted state transitions explicit and transactional.
- Keep research fixtures deterministic and version-controlled.
- Do not allow a debugging or convenience path to bypass Promotion Controller.

23. Testing Architecture



23.1 Unit Tests

- Pydantic/domain schema validation.
- Terraform normalization.
- Canonicalization/hashing.
- Identity classification.
- Dependency edge generation.
- Impact/invalidation rules.
- Verifier predicates.
- Promotion decision matrix.
- State transition invariants.

23.2 Integration Tests

- Terraform fixture → normalized model.
- Normalized model → dependency graph.
- Graph + state → impact report.
- Candidate → verification report.

- Candidate + report → promotion decision.
- Persistence round-trip.

23.3 Adversarial Tests

- Broken protected functional invariant after security fix.
- Deleted dependency.
- Resource replacement.
- Identity ambiguity.
- Dependency ambiguity.
- Unsupported resource/attribute.
- Verifier UNKNOWN.
- Verifier error.
- Malformed LLM patch.
- Attempt to mutate trusted state through candidate path.

24. Canonical Fixed-Patch Vertical Slice

24.1 Scenario A — Regression Must Be Rejected

```

Trusted State
├─ Security: public SSH problem
└─ Functional invariant: EC2 → RDS TCP/5432 works

Fixed Patch
└─ removes public SSH
   but breaks EC2 → RDS TCP/5432

Expected:
Security = PASS
Functional = FAIL
Decision = REJECT
Trusted State = unchanged

```

24.2 Scenario B — Safe Promotion

```

Trusted State
├─ Security: public SSH problem
└─ Functional invariant: EC2 → RDS TCP/5432 works

Fixed Patch
└─ removes public SSH
   while preserving EC2 → RDS TCP/5432

Expected:
Security = PASS
Functional = PASS
Decision = PROMOTE
Trusted State = candidate becomes v(N+1)

```

25. Implementation Roadmap

Milestone	Implementation	Exit criterion
M1	Project foundation + normalized Terraform model	Deterministic normalized representation from controlled fixtures.
M2	Trusted State + Invariant Store	State/invariant lifecycle works with persistence.
M3	Resource Identity	Identity/change classifications pass deterministic tests.
M4	Dependency Graph	Supported typed graph is deterministic and traceable.
M5	Impact/Invalidation	Affected-set calculations pass canonical and adversarial tests.
M6	Verification	Layered verifiers return explicit result semantics.
M7	Promotion Controller	Promotion safety matrix passes.
M8	Fixed-patch DAIL	Both canonical reject/promote scenarios work end-to-end.
M9	LLM integration	LLM produces untrusted candidate patches through adapter boundary.
M10	Evidence pipeline	Complete transition chain is persisted.
M11	Experiment harness	A/B/C can run under controlled protocol.
M12	Full testing + independent oracle	Research-critical safety and oracle tests pass.
M13	Pilot	Controlled end-to-end pilot produces complete evidence.
M14	Protocol freeze	Experiment protocol/configuration is frozen.
M15	Main experiment preparation	Reproducible experiment package ready.

26. M1 Detailed Implementation Plan

26.1 M1 Deliverables

16. Initialize Python 3.12 project and dependency management.
17. Create package/module skeleton.
18. Create base configuration mechanism.
19. Create Pydantic domain schemas for normalized resources and related metadata.
20. Create controlled Terraform fixtures for the MVP resource types.
21. Implement Terraform invocation wrapper.
22. Implement normalized resource extraction.
23. Implement canonical attribute serialization.
24. Implement deterministic resource fingerprinting.
25. Normalize security-group/security-group-rule representations within the supported scope.
26. Create M1 unit/integration tests.
27. Document unsupported constructs and normalization decisions.

26.2 M1 Non-Goals

- No LLM.

- No promotion.
- No full dependency engine.
- No AWS runtime deployment.
- No A/B/C experiment.
- No UI.

26.3 M1 Acceptance Criteria

- Fresh checkout can install the project reproducibly.
- All M1 tests pass.
- Same fixture produces the same normalized representation across repeated runs.
- Unsupported constructs are explicit.
- Resource identities/fingerprints are deterministic.
- No code path can mutate Trusted State because Trusted State functionality is not yet implemented.

27. Development Workflow

1. Select milestone
2. Confirm dependencies
3. Implement smallest domain contract
4. Write unit tests
5. Implement component
6. Run integration tests
7. Run regression suite
8. Inspect evidence/output
9. Review research impact
10. Update technical documentation
11. Commit milestone
12. Only then begin next dependent milestone

A milestone is not complete merely because code runs. It is complete when its implementation, tests, documentation, and research-validity implications are understood.

32.CI Pipeline

```

Pull Request
  ↓
Environment/setup
  ↓
Formatting/lint checks
  ↓
Schema/type validation
  ↓
Unit tests
  ↓
Integration tests
  ↓
Regression tests
  ↓
Artifact/test report

```

AWS-backed integration tests should be separated from the default deterministic CI path and executed only in controlled environments with explicit credentials and cost/safety controls.

29. Experiment Architecture

29.1 A/B/C

Condition	Persistent state	Verification policy
A — Stateless	No persistent DAIL protection state	Stateless baseline.
B — Stateful Full	Persistent invariant state	Full protected-set verification.
C — DAIL	Persistent invariant state	Dependency-aware selective verification.

The same core verification machinery should be reused where appropriate. The conditions must not receive unfairly different model capability or context. The intended experimental difference is the state/verification policy, not arbitrary prompt quality.

29.2 Research Run Trace

```
Experiment
→ Run
→ Attempt
→ Candidate
→ DAIL analysis
→ Decision
→ Independent ground truth
→ Metrics
```

30. Independent Ground Truth / Oracle

DAIL must not be evaluated solely by the verifier that DAIL itself uses. The oracle should be an independently implemented or independently executed evaluation path appropriate to each property.

- Keep oracle code/configuration separate from DAIL decision logic.
- Where runtime validation is used, run it in a controlled isolated AWS environment.
- Do not feed DAIL's promotion decision into the oracle.
- Record oracle version and input fingerprint.
- Use oracle disagreement as a research signal, not something to silently resolve.

31. Metrics and Evidence Engineering

31.1 Core metrics to support

- Repair success.
- Unsafe promotion rate.
- Regression detection.
- Verification work.
- Number of invariants verified.
- Number of invariants affected.
- Attempts per successful repair.
- UNKNOWN/UNSUPPORTED frequency.
- Oracle agreement/disagreement.
- Latency/cost where relevant.

The exact unit for ‘verification work’ must be fixed before the main experiment. It should not be chosen after seeing results.

32. Engineering Risks and Mitigations

Risk	Severity	Mitigation
Incorrect resource identity	Critical	Conservative identity rules, uncertainty states, adversarial tests.
Missed dependency	Critical	Typed graph, provenance, canonical dependency fixtures, independent oracle.
UNKNOWN treated as PASS	Critical	Explicit result types and promotion guard tests.
Rejected candidate becomes trusted	Critical	Single mutation boundary and transaction tests.
Verifier circularity	High	Independent oracle implementation/execution.
Terraform semantic mismatch	High	Terraform CLI authority + normalized model.
Scope expansion	High	Resource allowlist and documented out-of-scope handling.
LLM nondeterminism	Medium	Deterministic adapter for core tests; record model/configuration in experiments.
Evidence loss	High	Append-oriented event model and evidence tests.
Cloud cost/safety	High	Controlled AWS account/environment, least privilege, cleanup automation.
Over-engineering	Medium	Milestone gates and minimal infrastructure until required.

33. Locked / Provisional / Recommended Classification

LOCKED

- DAIL research contribution and core mechanism.
- Trusted State and protected invariant concepts.
- Candidate/LLM trust boundary.
- Identity, dependency, impact, verification, promotion responsibilities.
- PASS/FAIL/UNKNOWN semantics and conservative uncertainty handling.

- MVP AWS/Terraform resource scope.
- A/B/C experimental structure.
- Independent ground-truth requirement.
- Fixed-patch deterministic core before LLM.

PROVISIONAL

- Exact functional verifier implementation.
- Exact independent oracle implementation.
- Exact runtime validation boundary.
- Exact persistence schema details.
- Exact graph metadata schema.
- Optional formal verification.
- Exact experiment infrastructure.

RECOMMENDED

- Python 3.12.
- uv + pyproject.toml.
- Pydantic v2.
- Terraform CLI plus limited HCL tooling.
- SQLite + SQLAlchemy.
- NetworkX.
- boto3.
- pytest.
- GitHub Actions.
- Provider-agnostic LLM adapter.

34. Items Requiring Audit Chat Review

Review item	Why it matters
Functional verification semantics	Determines whether static modeling and/or runtime validation supports the intended functional claim.
Independent oracle design	Prevents circular evaluation.
Identity boundaries	Prevents hidden assumptions from invalidating protected evidence.
Verification-work metric	Ensures A/B/C efficiency comparison is well-defined before results.
A/B/C prompt/context equivalence	Prevents unfair advantage to DAIL.
Unsupported/unknown policy	Ensures uncertainty is treated consistently and defensibly.

These are not reasons to stop M1 engineering. They are research-critical decisions that must be settled before the affected later milestones or main experiment.

35. Definition of Done — Engineering MVP

- Supported Terraform subset is normalized deterministically.
- Trusted State and Candidate State are isolated and versioned.
- Protected invariants are evidence-backed.
- Identity classifications are deterministic for supported cases and conservative for ambiguous cases.
- Supported dependency graph is deterministic and traceable.

- Impact engine produces affected invariant sets.
- Verification engine produces explicit result semantics.
- Promotion Controller is the only trusted-state mutation authority.
- Canonical regression scenario is rejected.
- Canonical safe scenario is promoted.
- Rejected candidates cannot become trusted.
- UNKNOWN cannot silently promote.
- Evidence covers the full transition chain.
- Independent oracle exists before main experiment.
- A/B/C harness can execute under a fixed protocol.
- Automated tests cover safety-critical behavior.
- Documentation matches implementation.
- A reproducible local run can be executed from a clean checkout.

36. Final Implementation Checklist

Stage	Ready?
Technical architecture defined	YES
Technology stack selected	YES — recommended baseline
Repository structure defined	YES
Domain boundaries defined	YES
Trust boundary defined	YES
Terraform scope defined	YES
M1 acceptance criteria defined	YES
Core testing strategy defined	YES
Research evidence architecture defined	YES
A/B/C engineering boundary defined	YES
Independent oracle requirement defined	YES
Functional verifier exact implementation	AUDIT REVIEW
Exact verification-work metric	AUDIT REVIEW
M1 coding	NEXT STEP

37. Final Engineering Directive

This document is the implementation baseline. Implementation should proceed from M1 forward and should not skip dependency gates merely to reach the LLM or UI faster.

When implementation exposes a contradiction, technical impossibility, or research-validity concern, do not silently alter the architecture. Stop at the affected boundary, document the issue, propose alternatives, and mark the item REQUIRES AUDIT CHAT REVIEW.

The engineering objective is not simply to produce a working application. It is to produce a deterministic, testable, auditable, reproducible implementation of the approved DAIL mechanism whose software behavior can later support scientifically defensible A/B/C evidence.

Appendix A — First Implementation Command Sequence

```
# Environment
python 3.12
uv sync
```

```
# Quality
uv run pytest

# M1 target
uv run python -m dail.terraform_model ...

# Full deterministic suite
uv run pytest tests/unit tests/integration tests/regression

# Later milestones
uv run pytest tests/e2e tests/adversarial
```

The exact commands and package entry points will be finalized when the repository is initialized; they are shown here as implementation intent, not as an already-created codebase.

Appendix B — Document Authority

This engineering report consolidates the uploaded CloudShield AI / DAIL implementation materials and the implementation-readiness decisions made from them. It is an engineering baseline, not a replacement for the underlying research record. Where a research-level decision is locked upstream, this report implements that decision rather than redefining it.